Full Stack Engineer - Internal Systems: Take-Home Challenge

Before you start

- Completion is not the end goal.
- Plan on about 2 to 3 hours. You do not need to spend more than that.
- You must use a coding agent while you work. Examples:
  - Claude Code
  - Codex
  - Cursor Agent
  - any similar tool
- If you do not use one, we will not continue with your application.
- We care about your planning and your design choices.
- We also care how you guide the agent.
- Do not try to build everything. A small slice that works beats a wide one that does not.
- Record your full screen and your face for the whole session.
- A Zoom meeting by yourself, recorded locally, works well.
- We want to see how you really work, not only the final result.

Scenario

- Our HR Manager runs hiring for five open roles.
- Every candidate answers six interview questions.
- HR reads every answer and scores it by hand from 1 to 4 against a grading guide.

This is not working:

- Scoring takes most of HR's week, so candidates wait more than a week for a decision.
- Strong candidates are lost to other offers while they wait.
- Two HR scorers often give the same answer different scores.
- Candidates keep emailing to ask whether their answers arrived.

What to build

Build a small internal web app. HR stays the decision-maker. The system suggests. HR decides.

You must build these four parts.

1. Candidate area (Next.js)

A candidate can:

- Open the interview from a personal link, with no account or password.
- See one question at a time. The questions are the six in the data pack.
- Answer with typed text or an uploaded audio file.
- Be told straight away that their answer arrived.
- Reload the page and come back to the same question.

2. HR area (Next.js)

HR can:

- Sign in. A mocked sign-in is fine if you label it.
- See the candidates for a role, with each answer and its suggested score.
- See the reason for each suggested score.
- Override a suggested score. HR must give a reason.

3. Python API and a database

- Build the backend in Python. FastAPI or similar is fine.
- The backend exposes a JSON API.
- The backend owns:

  - the data
  - the business rules
  - the permissions
- Use a database of your choice. Examples:
  - SQLite
  - Postgres
  - Firestore
- In-memory storage is not enough for this challenge.

4. Scoring as a background job, with a real language model

- A background job scores each answer from 1 to 4 against its question's grading guide.
- The scorer must call a language model.
- You may use any model API you have access to.
- Validate what the model returns before you store it.
- The HR area shows the status of every job. The statuses are:
  - queued
  - scored
  - failed
  - needs a person
- A recorded or cached model response is not acceptable as the working app.
- In your tests you may replace the model client with a fake. That is fine.

Constraints

- Use Python for the backend.
- Use TypeScript or JavaScript for the frontend.
- Use any styling you like. We grade clarity, not polish.
- Mocked email, transcription and sign-in are fine if you label them clearly.
- A mocked or stub scorer is not fine.

What must be true

- Submitting the same answer twice does not create two scores. HR sees which one counts.
- A candidate can never see any of these, even by calling the API directly:
  - grades
  - other candidates
  - HR pages
- A failed scoring job is visible to HR, with a retry. It is never shown as a low score.
- An answer that tries to instruct the scorer cannot change its score.
- The typed answers in the data pack can be loaded and scored.
- The app runs from your README on a clean machine.

Audio uploads

- The candidate area must accept an uploaded audio file and store it.
- HR must be able to see that an audio answer arrived.
- Transcribing and scoring audio is optional. See the extras below.

Optional extras

Only try these after the required parts work. Skipping all of them is fine.

- Report how closely your suggested scores agree with HR's scores in hr_reference_scores.csv.
- Transcribe and score the five audio-only answers in the data pack. A mocked transcript is fine if labelled.
- A list of answers that need a person, each with the reason.
- A candidate list ordered to help HR decide who to read first.
- A consent screen and a microphone check in the candidate area.
- Clear error messages and a retry in the candidate area.

- A deployed demo.

The data

- Download the data pack: https://github.com/Growtrics/hiring-challenge-data/releases/latest/download/business-systems-pack.zip
- The README inside the pack explains each file.
- You need two parts of it: questions.md and the answers folder.
- questions.md holds the six questions with their grading guides.
- The answers folder holds 35 answers. Five of them are audio only.
- All of the data is made up for this challenge.
- You may add your own test data.

What we look at

- Product: does your slice work end to end, and would HR find it useful?
- Design: are the API and the data model sensible? Is the frontend and backend split clear?
- Reliability: do duplicates and failures end in a clear state? Is the model's output checked? We weigh this most.
- Your use of the coding agent: do you plan first? Do you read and correct what it wrote?
- Ownership: can you explain your own code and your choices?

Red flags

- The frontend shows data the API should never have sent. Permissions exist only in the browser.
- A resubmission or a double click creates duplicate records.
- The README does not run.
- The walkthrough replays a saved result instead of the running app.
- The scorer returns saved or fixed results instead of calling a language model.
- Agent output is pasted in unread, or you cannot explain code in your own repository.
- A mock is presented as a live result.

Deliverables

Send back exactly two things.

1. Your code: a private GitHub repository.
    - Share it with praveen.k@growtrics.ai and wayne.le@growtrics.ai.
    - Keep the commit history intact.
    - Include a short README.md.
    - Say how to set up the app.
    - Say how to run it.
    - Say how to test it.
        - Include a one-page architecture note. It covers:
            - the data model
            - the API endpoints
            - how an answer moves from submission to a score
            - what you chose not to build
        - Leave out dependency folders and build output.
2. Your recording: a Google Drive link to your full screen and face throughout the session.
    - Share it with hr-submissions@talotrace.ai as Viewer.
    - End the recording with a walkthrough of both areas handling the data pack's answers.
    - Do not send the walkthrough separately.

Reply to the invitation email with both links.

Again, completion is not the point.

- We care most about the slice you chose.
- We also care how well you used a coding agent.

Thank you for your time. We look forward to seeing how you work.